

Ciência de Dados para Análise de Desempenho de Aplicações HPC

Lucas Mello Schnorr, UFRGS

Escola Regional de Alto Desempenho Região Sudeste
Universidade Federal de Juiz de Fora (UFJF)
Juiz de Fora - MG, 12 de agosto de 2026



Propaganda

Quem sou eu?

Lucas Mello Schnorr

Professor no INF/UFRGS

Orientador Mestrado e Doutorado

<http://www.inf.ufrgs.br/~schnorr>

schnorr@inf.ufrgs.br



Formação

Ciência da Computação

- Graduação UFSM (2003)
- Mestrado UFRGS (2005)
- Doutorado UFRGS co-tutela no INPG, Grenoble, França (2009)
- Pós-doutorado no CNRS (2010 – 2012)
- Pós-doutorado no Inria (2016/2017)

Porto Alegre



About Porto Alegre: <http://www.youtube.com/watch?v=pCh4h1aYxtw>



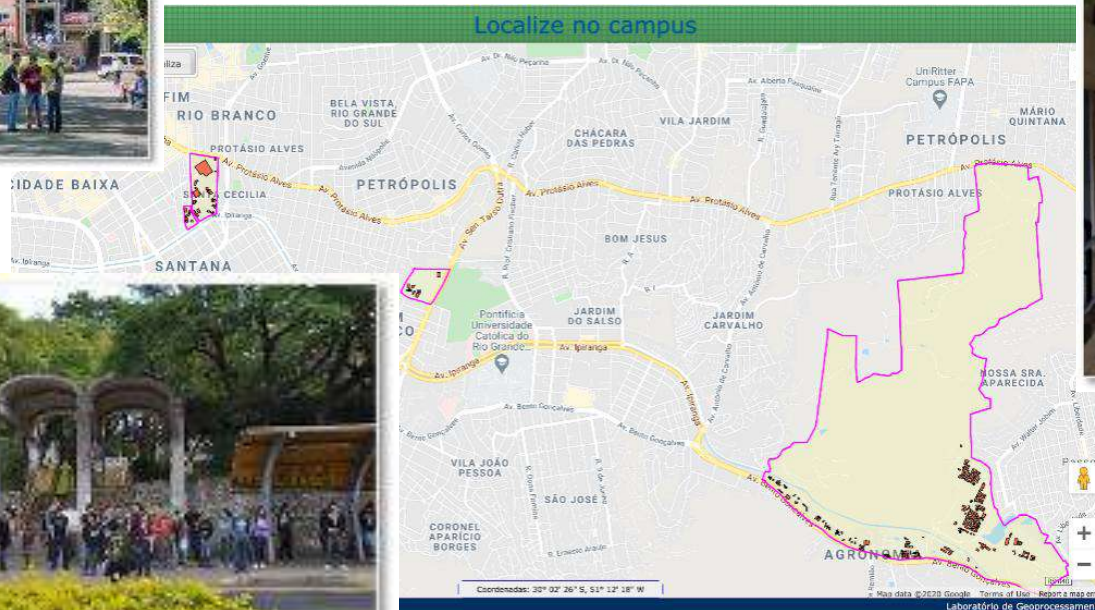
Alguns dados sobre a UFRGS

- A UFRGS foi fundada em 1934
 - Primeiro curso em 1895
- Uma das melhores universidades federais do Brasil, de acordo com o MEC
- Uma das **top-5 universidades do Brasil** por diversos rankings





UFRGS: Campus do Vale



Instituto de Informática





Programa de Pós-Graduação em Computação (PPGC)

Site do programa: <http://www.inf.ufrgs.br/ppgc/>



Oferece

- Mestrado, entradas semestrais
 - Para quem quer bolsa, fazer o Poscomp da SBC
 - Bolsas por projeto, entrar em contato com professores
- Doutorado, entrada com fluxo contínuo

Fatos marcantes

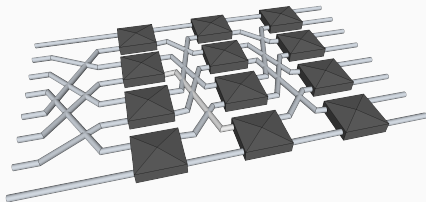
- Conceito máximo (7) pela CAPES
- Internacionalização da formação e da investigação

GPPD - Grupo de Processamento Paralelo e Distribuído

Site do grupo de pesquisa:

<http://www.inf.ufrgs.br/gppd/site/>

Logo do grupo de pesquisa



Eixos principais de investigação

- High Performance Computing
- Computer Architecture
- Performance Analysis
- Cloud Computing

Parque Computacional de Alto Desempenho (PCAD)

URL: <http://pcad.inf.ufrgs.br/>

Possui ≈ 50 nós, $\approx 1000+$ núcleos de CPU e $\approx 100000+$ de GPU



Localização: *Data Center* do Instituto de Informática, UFRGS

Temos um GT (Grupo de Trabalho) → DevOps para HPC @ ERAD/RS 2023

- Formamos alunos no gerenciamento destas plataformas
- SLURM, NFS, LDAP, ...

Introdução

Análise de Desempenho em HPC: objetivos típicos

Objetivos principais da análise de desempenho

Análise de Desempenho em HPC: objetivos típicos

Objetivos principais da análise de desempenho

Compreender o comportamento da aplicação

- Identificar gargalos de desempenho (CPU, memória, rede, I/O)
- Entender padrões de comunicação entre processos e threads
- Avaliar o uso dos recursos computacionais disponíveis

Análise de Desempenho em HPC: objetivos típicos

Objetivos principais da análise de desempenho

Compreender o comportamento da aplicação

- Identificar gargalos de desempenho (CPU, memória, rede, I/O)
- Entender padrões de comunicação entre processos e threads
- Avaliar o uso dos recursos computacionais disponíveis

Guiar a otimização

- Direcionar esforços para as partes de maior impacto (*hotspots*)
- Validar o efeito de modificações no código
- Apoiar decisões de escalonamento e configuração de execução

Análise de Desempenho em HPC: objetivos típicos

Objetivos principais da análise de desempenho

Compreender o comportamento da aplicação

- Identificar **gargalos** de desempenho (CPU, memória, rede, I/O)
- Entender padrões de **comunicação** entre processos e threads
- Avaliar o uso dos **recursos computacionais** disponíveis

Guiar a otimização

- Direcionar esforços para as partes de **maior impacto** (*hotspots*)
- Validar o efeito de **modificações** no código
- Apoiar decisões de **escalonamento** e configuração de execução

Garantir qualidade e reprodutibilidade

- Produzir análises **reprodutíveis** e verificáveis
- Comparar desempenho entre diferentes **plataformas**
- Gerar evidências para **publicações** científicas

Análise de Desempenho em HPC: o problema

Análise de desempenho de aplicações HPC é **essencial** para uso eficiente dos recursos

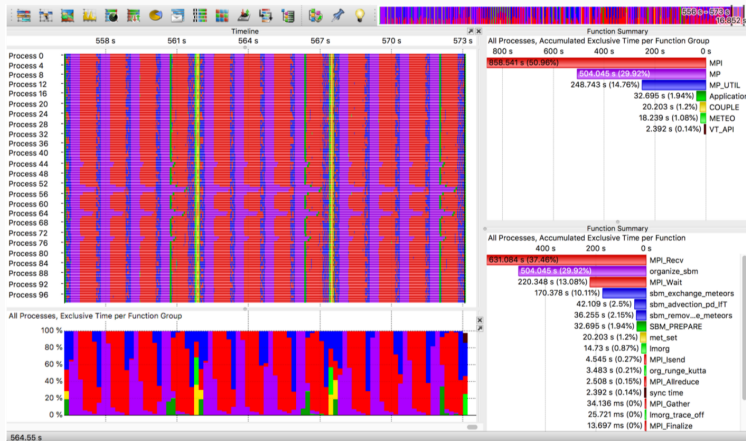
Análise de Desempenho em HPC: o problema

Análise de desempenho de aplicações HPC é **essencial** para uso eficiente dos recursos

Abordagem tradicional: ferramentas **monolíticas**

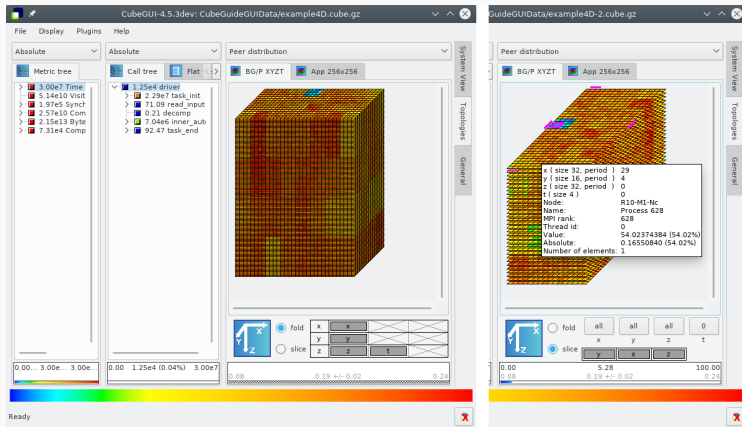
- Pouca capacidade de **expressão** analítica
- Baixa capacidade de **extensão** e adaptação
- Dificuldade de integração com fluxos modernos de análise

Alguns exemplos: Vampir 1/4



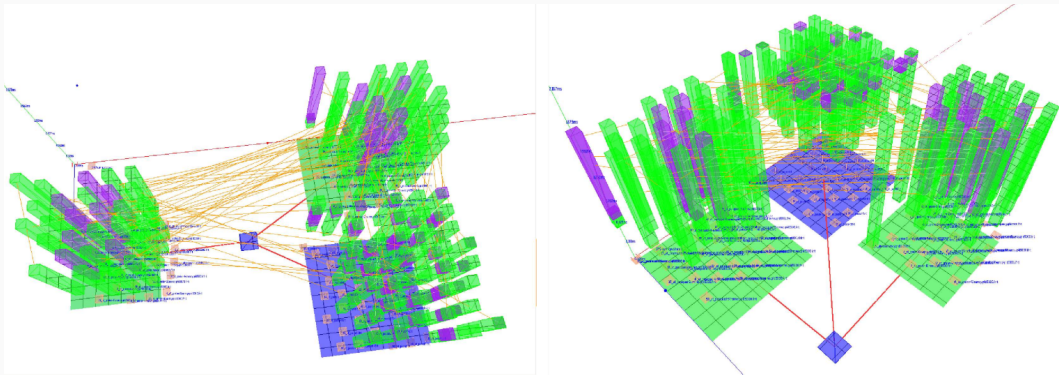
Williams, William, and Holger Brunst. "Parallel performance engineering using Score-P and Vampir." Companion of the 2023 ACM/SPEC International Conference on Performance Engineering. 2023.

Alguns exemplos: Scalasca Cube 2/4



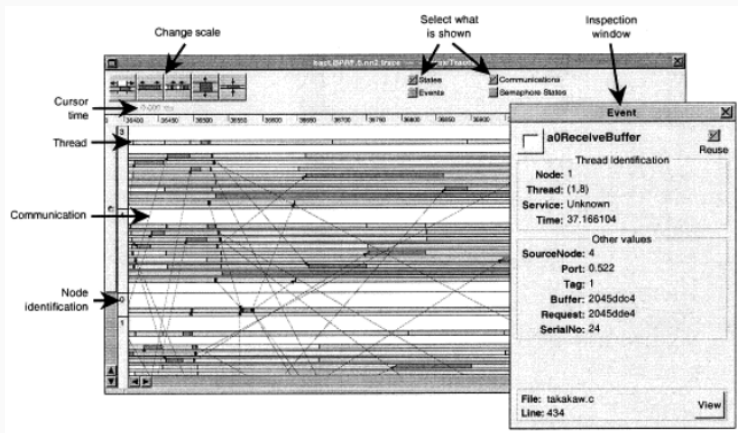
Zhukov, Ilya, et al. "Scalasca v2: Back to the future." Tools for High Performance Computing 2014: Proceedings of the 8th International Workshop on Parallel Tools for High Performance Computing, October 2014, HLRS, Stuttgart, Germany. Cham: Springer International Publishing, 2015.

Alguns exemplos: Triva 3/4



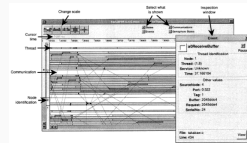
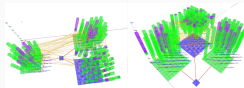
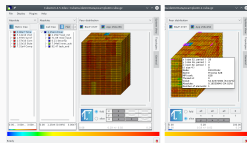
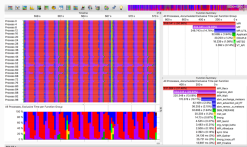
Schnorr, Lucas Mello, Guillaume Huard, and Philippe OA Navaux. "Triva: Interactive 3d visualization for performance analysis of parallel applications." *Future Generation Computer Systems* 26.3 (2010): 348-358.

Alguns exemplos: Paje 4/4



Flexible performance visualization of parallel and distributed applications. J. Chassin de Kergommeaux, B. de Oliveira Stein
Future Generation Computer Systems 19 (2003) 735-747.

Análise de Desempenho em HPC: as consequências



- Análise limitada de rastros de execução e perfis de desempenho
- Ciclo de otimização lento
- Dificuldade de realizar automações
 - Por exemplo via LLMs ou SLMs
 - Ou até via métodos tradicionais de código de análise
- Pouco reproduzível
 - Não temos o “código fonte” de cada visão específica

Ciência de Dados como Solução

Surgimento de ferramentas de ciência de dados e big data

Ciência de Dados como Solução

Surgimento de ferramentas de **ciência de dados** e **big data**

Novas possibilidades

- Análise expressiva e flexível de **rastros de execução**
- Suporte a uma variedade gigante de tipos de aplicações
 - Tradicional Bulk-Synchronous Parallel (BSP, MPI, MPI+X)
 - Baseadas em **tarefas** (StarPU, Parsec, OpenMP tasks)
- Tratamento de desafios de **escalabilidade**
 - Enorme volume de dados rastreados

Ciência de Dados como Solução

Surgimento de ferramentas de **ciência de dados** e **big data**

Novas possibilidades

- Análise expressiva e flexível de **rastros de execução**
- Suporte a uma variedade gigante de tipos de aplicações
 - Tradicional Bulk-Synchronous Parallel (BSP, MPI, MPI+X)
 - Baseadas em **tarefas** (StarPU, Parsec, OpenMP tasks)
- Tratamento de desafios de **escalabilidade**
 - Enorme volume de dados rastreados

Além da análise de desempenho

- **Reprodutibilidade** dos experimentos e análises
- **Transparência científica** nos resultados

1. Revisar o funcionamento do paralelismo de tarefas
 - *Sequential Task Flow* (STF) com o runtime StarPU
2. Apresentar o pacote R `StarVZ`
 - R-Based Visualization Techniques for Task-Based Applications
 - <https://cran.r-project.org/web/packages/starvz/>
3. Alguns exemplos recentes de uso

StarVZ

Paralelismo de Tarefas no StarPU

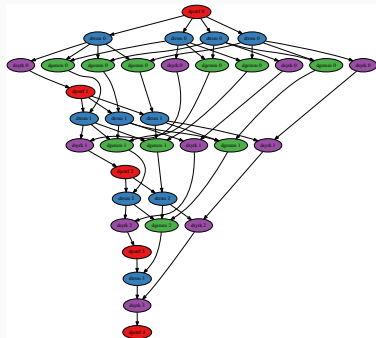
Aplicação é um grafo de tarefas → Paradigma *Sequential Task Flow* – STF

Exemplo: Fatorização de cholesky por blocos

```

for (k = 0; k < N; k++) {
    DPOTRF(RW,A[k][k]);
    for (i = k+1; i < N; i++)
        DTRSM(RW,A[i][k], R,A[k][k]);
    for (i = k+1; i < N; i++) {
        DSYRK(RW,A[i][i], R,A[i][k]);
        for (j = k+1; j < i; j++)
            DGEMM(RW,A[i][j],
                R,A[i][k], R,A[j][k]);
    }
}

```

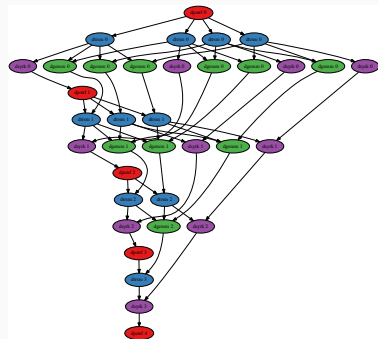


Paralelismo de Tarefas no StarPU

Aplicação é um grafo de tarefas → Paradigma *Sequential Task Flow* – STF

Exemplo: Fatorização de cholesky por blocos

```
for (k = 0; k < N; k++) {  
    DPOTRF(RW, A[k][k]);  
    for (i = k+1; i < N; i++)  
        DTRSM(RW, A[i][k], R, A[k][k]);  
    for (i = k+1; i < N; i++) {  
        DSYRK(RW, A[i][i], R, A[i][k]);  
        for (j = k+1; j < i; j++)  
            DGEMM(RW, A[i][j],  
                R, A[i][k], R, A[j][k]);  
    }  
}
```



Um *runtime* (StarPU) é responsável pelo escalonamento das tarefas

- Respeita as dependências, faz dados trafegarem

StarVZ: visualização de rastros de paralelismo de tarefas

Características

- Análise *post-mortem*, após a finalização da aplicação paralela
- Possui uma série de painéis de visualização
- Totalmente baseado no framework de análise `R/tidyverse`

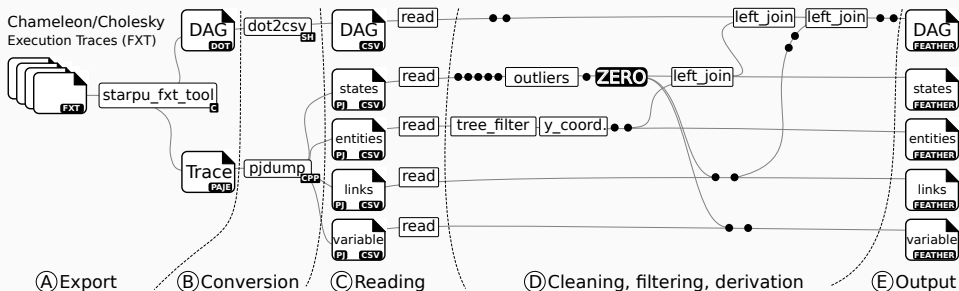


- Coleção de pacotes → ciência de dados
- Compartilham filosofia e gramática

A Visual Performance Analysis Framework for Task-based Parallel Applications running on Hybrid Clusters, Vinicius Garcia Pinto, Lucas Mello Schnorr, Luka Stanisic, Arnaud Legrand, Samuel Thibault, Vincent Danjean, in *Concurrency and Computation: Practice and Experience*, Wiley, 2018, 30 (18), pp.1-31.

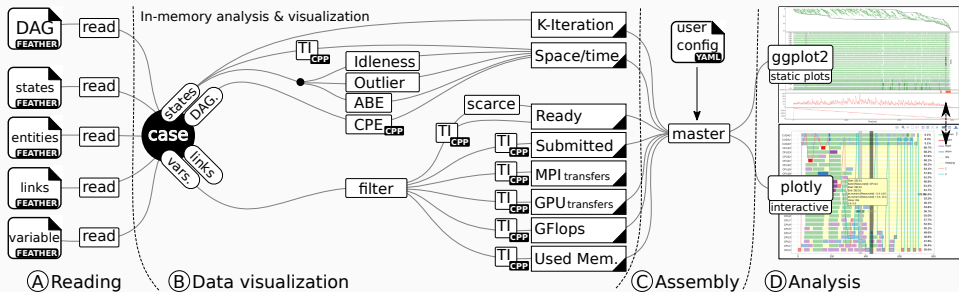
StarVZ: Phase 1

1. Aplicação StarPU é executada, gera arquivos `prof_file_X`
 - Arquivos binários (preocupação com baixa intrusividade)
2. Executamos `starpu_fxt_tool`, ferramenta para exportar
3. Executamos então a fase #1 do StarVZ
 - No cluster → gerar arquivos de leitura rápida



StarVZ: Phase 2

1. Dados são normalmente trazidos para o Laptop
2. Leitura rápida dos dados em formato `parquet`
3. Painéis de visualização, opção de lidar direto com as tabelas
 - Construir seu próprio *script* de análise



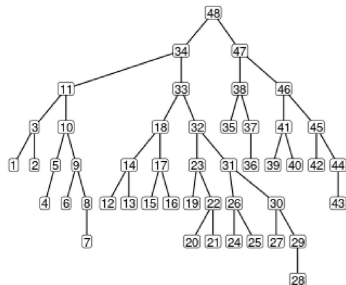
Exemplos de uso

Fatoração de matrizes esparsas QR-Mumps 1/2

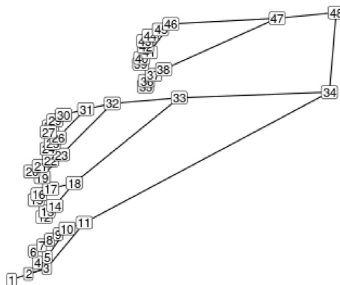
https://qr_mumps.gitlab.io/

Árvore de eliminação para guiar o processo de fatoração

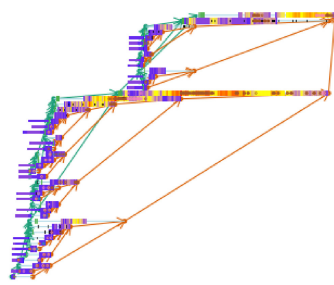
A) Elimination Tree



B) Y is Submission Order, X is Time

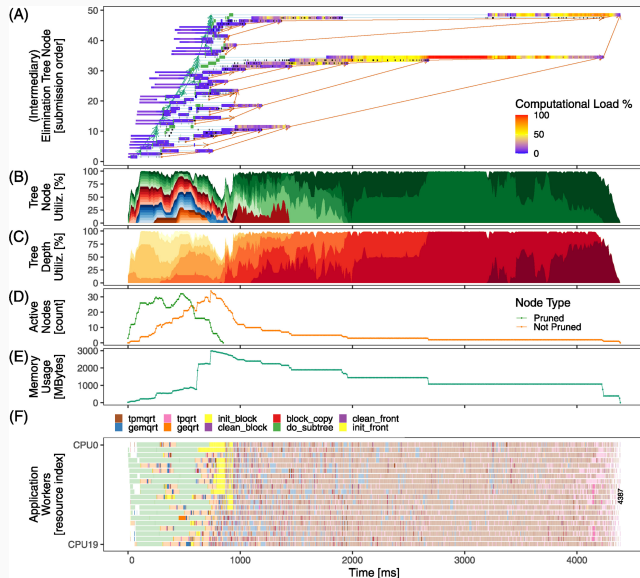


C) Our Visualization



Miletto, Marcelo Cogo, et al. "Performance analysis of task-based multi-frontal sparse linear solvers: Structure matters." *Future Generation Computer Systems* 135 (2022): 409-425.

Fatoração de matrizes esparsas QR-Mumps 2/2

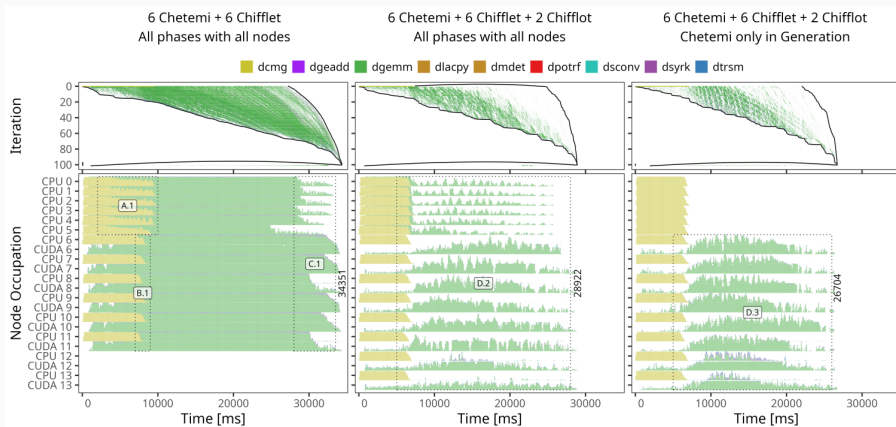


Seis painéis de análise

1. Processamento da árvore de eliminação
2. Intensidade de processamento por nó da árvore
3. Intensidade de processamento por profundidade da árvore
4. Quantidade de nós ativos
5. Quantidade de memória utilizada
6. Comportamento dos trabalhadores [cores são os tipos de tarefas]

Aplicações em cluster com ExaGeoStat sobre StarPU-MPI

<https://github.com/ecrc/exageostat>



Asynchronous multi-phase task-based applications: Employing different nodes to design better distributions. Future Gener.

Comput. Syst. 147: 119-135 (2023)

Conclusão

Análise de desempenho mais específica por aplicação

- Sobreposição comunicação/cálculo (MPI, CUDA)
- Identificação das razões de ociosidade
- Identificar atrasos de escalonamento
- Dar liberdade ao analista de testar cenários *what-if*

Conclusões

Análise de desempenho mais específica por aplicação

- Sobreposição comunicação/cálculo (MPI, CUDA)
- Identificação das razões de ociosidade
- Identificar atrasos de escalonamento
- Dar liberdade ao analista de testar cenários *what-if*

Separação clara entre o rastreamento e a análise

- Usando formatos e ferramentas abertas
- Permitir emprego de técnicas de ciência de dados
 - Formatos eficientes de leitura *in-memory*
 - Usar frameworks de aceleração (RAPIDS, PySpark)

Conclusões

Análise de desempenho mais específica por aplicação

- Sobreposição comunicação/cálculo (MPI, CUDA)
- Identificação das razões de ociosidade
- Identificar atrasos de escalonamento
- Dar liberdade ao analista de testar cenários *what-if*

Separação clara entre o rastreamento e a análise

- Usando formatos e ferramentas abertas
- Permitir emprego de técnicas de ciência de dados
 - Formatos eficientes de leitura *in-memory*
 - Usar frameworks de aceleração (RAPIDS, PySpark)

Favorecer emprego de LLMs/SLMs

- Construção de code snippets de análise

Obrigado pela atenção!

`schnorr@inf.ufrgs.br`

